

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..11

FILE UPLOADS

 Copy as Markdown

File uploads

Let your users upload avatars, documents, or any other files. The app sends the file straight to storage instead of through your endpoint methods, which keeps large files out of your API calls. Out of the box, your server stores files in the database, which works well for development. In production, configure Google Cloud Storage, AWS S3, or Cloudflare R2 instead. To keep files on local disk or a NAS, see [Custom cloud storage](#).

Upload a file

A `public` and a `private` file storage are set up by default. You can replace these or add more configurations for other file storages.

Server-side code

There are a few steps required to upload a file. First, you must create an upload description on the server and pass it to your app. The upload description grants access to the app to upload the file. If you want to grant access to any file, you can add the following code to one of your endpoints. However, in most cases, you may want to restrict which files can be uploaded.

```
Future<String> getUploadDescription(Session session, String path) async {  
  return await session.storage.createUploadDescription(  
    storageId: 'public',  
    path: path,  
  );  
}
```

The `createUploadDescription` method also accepts an optional `UploadOptions` object to control the upload:

- `UploadOptions.expirationDuration`: How long the upload URL is valid. Defaults to 10 minutes.
- `UploadOptions.maxFileSize`: Maximum allowed file size in bytes. Defaults to 10 MiB.



Ask AI

- `UploadOptions.contentLength`: The exact file size in bytes. When provided, the storage provider validates the upload size against `maxFileSize`.
- `UploadOptions.preventOverwrite`: When `true`, the upload will fail if a file already exists at the given path. Defaults to `false`.
- `UploadOptions.metadata`: HTTP metadata and custom key-value data to store with the file.

Upload metadata is represented by `FileMetadata`. It supports `contentType`, `cacheControl`, `contentDisposition`, `contentEncoding`, and a `custom` map. Serverpod includes these values in the signed upload request so the storage provider can attach them to the file.

```
Future<String> getRestrictedUploadDescription(
  Session session,
  String path,
  int fileSize,
) async {
  return await session.storage.createUploadDescription(
    storageId: 'public',
    path: path,
    options: UploadOptions(
      maxFileSize: 50 * 1024 * 1024, // 50 MB
      contentLength: fileSize,
      preventOverwrite: true,
      metadata: const FileMetadata(
        contentType: 'image/jpeg',
        cacheControl: 'public, max-age=3600',
        custom: {'source': 'profile-image'},
      ),
    ),
  );
}
```

An option may not be supported by every storage provider. Serverpod throws an exception when an unsupported or invalid option is used.

After the file is uploaded, verify that the upload completed. With a third-party service such as S3 or Google Cloud Storage, this is the only way to know it was not canceled.

```
Future<bool> verifyUpload(Session session, String path) async {
  return await session.storage.verifyUpload(
    storageId: 'public',
    path: path,
  );
}
```

Client-side code

To upload a file from the app side, first request the upload description. Next, upload the file, from either a `Stream` or a `ByteData` object. When uploading from a `Stream`, pass the file length if you know it:

without a length, a multipart upload buffers the whole file in memory. The uploader does not report upload progress. On failure it returns `false` with no status code and no exception, so a network error and a file that is too large look the same. Finally, verify the upload with the server.

```
final uploadDescription = await client.myEndpoint.getUploadDescription('myfile');
final uploader = FileUploader(uploadDescription);
final uploaded = await uploader.upload(myStream, myFileLength);
final success = await client.myEndpoint.verifyUpload('myfile');
```

INFO

In a real-world app, you most likely want to create the file paths on your server. For your file paths to be compatible with S3, do not use a leading slash. Only use standard characters and numbers. For example:

```
'profile/$userId/images/avatar.png'
```

Size limits

When the app uploads through your API server (the default database storage), two size limits apply. The upload uses the smaller of the two.

Limit	Default	Set in
<code>maxRequestSize</code>	524288 (512 KiB)	<code>config/<run-mode>.yaml</code> or <code>SERVERPOD_MAX_REQUEST_SIZE</code>
<code>UploadOptions.maxFileSize</code>	10 MiB	<code>createUploadDescription</code>

A 5 MB file with default config is rejected even though `maxFileSize` is 10 MiB. Raise `maxRequestSize` in every run-mode YAML you use to at least the largest file you accept. See the [Configuration reference](#).

Uploads to S3, Google Cloud Storage, and R2 go to the provider, so `maxRequestSize` does not apply to the file body.

Access stored files

You can check if a file exists or retrieve it directly from your server. Files in public storage are also accessible via URL.

To check if a file exists, use the `fileExists` method.

```
final exists = await session.storage.fileExists(  
  storageId: 'public',  
  path: 'my/file/path',  
);
```

If the file is in a public storage, you can access it through its URL.

```
final url = await session.storage.publicDownloadUrl(  
  storageId: 'public',  
  path: 'my/file/path',  
);
```

To access public URLs for many files at once, use `publicDownloadUrls`.

```
final urls = await session.storage.publicDownloadUrls(  
  storageId: 'public',  
  paths: const ['my/file/path', 'my/other-file/path'],  
);
```

A private storage can provide temporary, signed URLs when you want an authorized user to download a specific file without making the whole storage public. First confirm that the caller is allowed to access the file. Then create a signed URL that grants temporary access to the file:

```
final url = await session.storage.temporaryDownloadUrl(  
  storageId: 'private',  
  path: 'my/private/file/path',  
  options: const TemporaryDownloadUrlOptions(  
    expirationDuration: Duration(minutes: 5),  
    downloadFileName: 'private.pdf',  
    contentType: 'application/pdf',  
  ),  
);
```

Anyone who has the signed URL can download that one file until the URL expires. Return it only after your endpoint has authenticated and authorized the caller. `downloadFileName` asks the browser to download the response under that name, and `contentType` overrides the response content type.

Use `statFile` to retrieve the file size, last-modified time, content headers, provider entity tag, and custom metadata:

```
final stat = await session.storage.statFile(  
  storageId: 'public',  
  path: 'my/file/path',  
);
```

```
);
```

You can also retrieve a file directly from your server:

```
final myByteData = await session.storage.retrieveFile(  
  storageId: 'public',  
  path: 'my/file/path',  
);
```

To store a file directly from the server, use `storeFile` with `StoreFileOptions`. You can set `preventOverwrite` to `true` to ensure the write fails if a file already exists at the given path.

```
await session.storage.storeFile(  
  storageId: 'public',  
  path: 'my/file/path',  
  byteData: myByteData,  
  options: const StoreFileOptions(  
    preventOverwrite: true,  
    metadata: FileMetadata(  
      contentType: 'application/pdf',  
      cacheControl: 'private, max-age=300',  
      custom: {'document-type': 'invoice'},  
    ),  
  ),  
);
```

`StoreFileOptions` also has an optional `expiration` timestamp. Storage providers that cannot expire individual files throw an exception when it is set.

To delete a stored file, use `deleteFile` with the same `storageId` and `path`.

Configure a storage provider

Each storage is identified by a `storageId`. Serverpod comes with two default storages, `public` and `private`. Replace these with a cloud-backed implementation, or add additional storages with custom IDs. Calling `pod.addCloudStorage` with `public` or `private` replaces that default. Call it before `pod.start()`. For local disk or a NAS, see [Custom cloud storage](#).

In new projects, `server.dart` already swaps both defaults for `ServerpodCloudProvider`, from the `serverpod_cloud_storage` package:

```
pod.addCloudStorage(  
  await ServerpodCloudProvider.private(  
    fallback: () => DatabaseCloudStorage('private'),  
  ),  
);
```

```
pod.addCloudStorage(  
  await ServerpodCloudProvider.public(  
    fallback: () => DatabaseCloudStorage('public'),  
  ),  
);
```

When Serverpod Cloud provides storage for your project, these calls connect to it. Otherwise, for example when you run the server locally, the `fallback` runs and files are stored in the database. To keep files somewhere else, replace these calls with one of the storages below.

Pick the package that matches your provider. Use `serverpod_cloud_storage_s3` for AWS S3, `serverpod_cloud_storage_gcp` for Google Cloud Storage, or `serverpod_cloud_storage_r2` for Cloudflare R2.

Configure Google Cloud Storage

The default GCP path uses HMAC keys and the S3-compatible API. Create a [service account HMAC key](#), then:

1. Grant the service account the Storage Admin role.
2. Create an HMAC key for that account under **Cloud Storage > Settings > Interoperability**.
3. When you create the bucket, set **Access control** to **Fine-grained** and turn off **Prevent public access**.

Add the HMAC keys to `config/passwords.yaml`, or pass them as `SERVERPOD_HMAC_ACCESS_KEY_ID` and `SERVERPOD_HMAC_SECRET_KEY`.

config/passwords.yaml

```
shared:  
  HMACAccessKeyId: 'XXXXXXXXXXXXXXX'  
  HMACSecretKey: 'XXXXXXXXXXXXXXXXXXXXXXXXXXXX'
```

Add the package and import it in `server.dart`.

```
dart pub add serverpod_cloud_storage_gcp
```

```
import 'package:serverpod_cloud_storage_gcp/serverpod_cloud_storage_gcp.dart'  
as gcp;
```

The `bucket` parameter is the GCP bucket name. Set `publicHost` if the bucket is reachable on a custom domain behind a load balancer. Use the bucket's GCS region, for example `us-central1`.

```
pod.addCloudStorage(  
  gcp.GoogleCloudStorage(  
    bucket: 'my-bucket',  
    publicHost: 'https://my-bucket.storage.googleapis.com',  
    region: 'us-central1',  
  ),  
);
```

```

serverpod: pod,
storageId: 'public',
public: true,
region: 'us-central1',
bucket: 'my-bucket-name',
publicHost: 'storage.myapp.com',
),
);

```

Use native Google Cloud Storage

As an alternative to HMAC keys, use Google Cloud Storage's native JSON API with a service account. This path supports custom metadata, conditional writes with `preventOverwrite`, and signed temporary download URLs. It lives in the same `serverpod_cloud_storage_gcp` package.

For a public storage, make the bucket itself publicly readable, because Serverpod doesn't make each uploaded file public. For example, turn on uniform bucket-level access and grant `allUsers` the Storage Object Viewer role.

The factory constructors are asynchronous, so create the storage before starting the pod:

```

pod.addCloudStorage(
  await gcp.NativeGoogleCloudStorage.create(
    serverpod: pod,
    storageId: 'public',
    public: true,
    bucket: 'my-bucket-name',
    publicHost: 'storage.myapp.com',
  ),
);

```

The `create` factory loads the service account JSON from `passwords.yaml` (key: `gcpServiceAccount`) or the environment variable `SERVERPOD_PASSWORD_gcpServiceAccount`:

config/passwords.yaml

```

shared:
  gcpServiceAccount:
    '{"type":"service_account","project_id":"...","private_key":"...",...}'

```

To pass the JSON directly instead, use `fromServiceAccountJson`:

```

pod.addCloudStorage(
  await gcp.NativeGoogleCloudStorage.fromServiceAccountJson(
    storageId: 'public',
    public: true,
    bucket: 'my-bucket-name',

```

```

    serviceAccountJson: myServiceAccountJson,
  ),
);

```

On Google Compute Engine or Cloud Run, use Application Default Credentials instead:

```

pod.addCloudStorage(
  await gcp.NativeGoogleCloudStorage.fromApplicationDefaultCredentials(
    storageId: 'public',
    public: true,
    bucket: 'my-bucket-name',
  ),
);

```

NOTE

When using Application Default Credentials, the service account must have the `iam.serviceAccounts.signBlob` IAM permission to generate signed URLs.

Configure AWS S3

Create an S3 bucket and an IAM user whose access is limited to that bucket. Avoid root-user access keys. Put CloudFront in front of the bucket if you want a custom domain and TLS certificate. Add the access keys to `config/passwords.yaml`, or pass them as `SERVERPOD_AWS_ACCESS_KEY_ID` and `SERVERPOD_AWS_SECRET_KEY`.

config/passwords.yaml

```

shared:
  AWSAccessKeyId: 'XXXXXXXXXXXXXX'
  AWSSecretKey: 'XXXXXXXXXXXXXXXXXXXXXXXXXXXX'

```

Add the package and import it in `server.dart`.

```
dart pub add serverpod_cloud_storage_s3
```

```

import 'package:serverpod_cloud_storage_s3/serverpod_cloud_storage_s3.dart'
  as s3;

```

Set `publicHost` if the bucket is accessible on a custom domain through CloudFront.

```
pod.addCloudStorage(
```

```
s3.S3CloudStorage(
  serverpod: pod,
  storageId: 'public',
  public: true,
  region: 'us-west-2',
  bucket: 'my-bucket-name',
  publicHost: 'storage.myapp.com',
),
);
```

Configure Cloudflare R2

R2 is S3-compatible and uses presigned PUT uploads. Create [R2 API tokens](#) in the Cloudflare dashboard, then add them to `config/passwords.yaml`, or pass them as `SERVERPOD_R2_ACCESS_KEY_ID` and `SERVERPOD_R2_SECRET_KEY`.

config/passwords.yaml

```
shared:
  R2AccessKeyId: 'XXXXXXXXXXXXXXXX'
  R2SecretKey: 'XXXXXXXXXXXXXXXXXXXXXXXXXXXX'
```

Add the package and import it in `server.dart`.

```
dart pub add serverpod_cloud_storage_r2
```

```
import 'package:serverpod_cloud_storage_r2/serverpod_cloud_storage_r2.dart'
  as r2;
```

Configure the storage with your Cloudflare account ID and bucket name:

```
pod.addCloudStorage(
  r2.R2CloudStorage(
    serverpod: pod,
    storageId: 'public',
    public: true,
    bucket: 'my-bucket-name',
    accountId: 'your-cloudflare-account-id',
    publicHost: 'storage.myapp.com',
  ),
);
```

Use a custom S3-compatible endpoint

The S3, GCP HMAC, and R2 packages are the ones to add by hand. They sit on `serverpod_cloud_storage_s3_compat`, which you can also use directly for another S3-compatible service such as LocalStack.

```
dart pub add serverpod_cloud_storage_s3_compat
```

```
import  
'package:serverpod_cloud_storage_s3_compat/serverpod_cloud_storage_s3_compat.dart';
```

Pass credentials yourself. `CustomEndpointConfig` sets the base URI.

`MultipartPostUploadStrategy` matches AWS-style POST uploads, for example LocalStack. Use

`PresignedPutUploadStrategy` for providers that only accept presigned PUT, such as R2.

```
pod.addCloudStorage(  
  S3CompatCloudStorage(  
    storageId: 'public',  
    public: true,  
    region: 'us-east-1',  
    bucket: 'my-bucket-name',  
    accessKey: pod.getPassword('MyAccessKeyId')!,  
    secretKey: pod.getPassword('MySecretKey')!,  
    endpoints: CustomEndpointConfig(  
      baseUrl: Uri.http('localhost:4566', '/'),  
      serviceName: 'LocalStack',  
    ),  
    uploadStrategy: MultipartPostUploadStrategy(),  
  ),  
);
```

Related

- [Custom cloud storage](#): local disk, NAS, and implementing `CloudStorage`.
- [Configuration](#): passwords file and environment variables for storage keys.
- [Sessions](#): the `storage` member used in the examples above.